



Ambiente CI/CD

INF 331 - Pruebas de Software

[Repositorio de Github](#)

Versión: 2.0

Fecha de creación: 24 de Octubre 2022

Última edición: 8 de Diciembre 2022

Fecha de validez: 9 de Diciembre 2022

Documento elaborado y revisado por:

Iñaki Oyarzún M.
201873620-1

Fernando Delgado
201804571-3

Luis Arturo González Arismendi
201892004-5

Introducción

Se busca desarrollar una aplicación web que se utilizará como recetario de cocina, las funcionalidades principales consisten en permitir al usuario administrar múltiples recetas desde un único menú.

Para esto se detallan los aspectos formales que hay que cumplir, en particular requerimientos funcionales y no funcionales. Por último se muestran de manera detallada los casos de uso para probar la aplicación con la suite creada que contiene uso de las herramientas Selenium, Jenkins, Slack, Jira y Github.

Análisis de requerimientos

La aplicación web está enfocada al usuario que busca de manera simple crear y mantener un recetario, en particular debe contar explícitamente con opciones para: "Agregar una receta", "Editar una receta", "Eliminar una receta" y "Ver detalles de una receta".

Por otro lado, es imperativo que la aplicación pueda ser probada de manera automatizada con la herramienta Jenkins en conjunto con Selenium, con la cual se elaborarán pruebas de casos de uso que permitan definir si la aplicación cumple con los requisitos funcionales y reglas de negocio definidas.

Supuestos

Se asumen los siguientes puntos con el fin de omitir detalles que no son relevantes para el desarrollo de la aplicación:

- Se asume que el entorno donde se desarrollarán las pruebas cuenta con los drivers necesarios para hacer uso completo de la suite integrada
- Se asume que en el campo para subir una imagen de la receta será usado exclusivamente para subir archivos de imagen (.jpg, .jpeg, .png).
- Se asume que el hardware donde se aloja la aplicación es apto para hacer de web server y puede cumplir con los requerimientos funcionales.

Requerimientos del negocio

Los requerimientos funcionales serán descritos de manera aislada tal que cada requerimiento pueda ser puesto a prueba de manera independiente.

Adicionalmente se describen qué atributos tiene que tener una receta válida:

- Título de la receta.
- Pasos a seguir de la receta.

- Ingredientes necesarios.
- Una foto de la receta ya preparada.

Requerimientos funcionales

Desde la aplicación debe ser posible realizar las siguientes tareas por cualquier usuario para cumplir con las especificaciones:

- El usuario debe poder agregar una receta.
- Todas las recetas agregadas deben ser visibles desde el menú principal.
- El usuario debe poder eliminar una receta.
- El usuario debe poder editar los datos de una receta.
- El usuario debe poder visualizar los detalles de una receta.

Requerimientos no funcionales

- La aplicación debe ser compatible con el navegador Chrome.
- La aplicación debe ser compatible con la librería de pruebas Selenium.
- Deben poder realizarse exitosamente pruebas automatizadas con Jenkins usando las pruebas de Selenium.
- Un usuario sin experiencia previa debe poder agregar una receta sin necesidad de ayuda externa.
- El sistema cuenta con testabilidad tal que se pueden realizar pruebas unitarias automatizadas para cada función.

Prioridad de requerimientos

Se utiliza el método MoScow para determinar la prioridad de los requerimientos funcionales y no funcionales establecidos.

- (M)ust: Obligatorio
- (S)hould: Alta prioridad
- (C)ould: Preferible pero no necesario
- (W)ould: Puede ser pospuesto para ejecución a futuro

Requerimiento	Prioridad
El usuario debe poder agregar una receta	M
Todas las recetas agregadas deben ser visibles desde el menú principal	M
El usuario debe poder eliminar una receta	M
El usuario debe poder editar los datos una receta	M
El usuario debe poder visualizar los detalles de una receta	M
La aplicación debe ser compatible con el navegador Chrome	M
La aplicación debe ser compatible con la librería de pruebas Selenium	M
Deben poder realizarse exitosamente pruebas automatizadas con Jenkins/Selenium	M
Un usuario sin experiencia previa debe poder agregar una receta sin necesidad de ayuda externa	S
El sistema cuenta con testabilidad tal que se pueden realizar pruebas unitarias automatizadas para cada función	M

Dependencias entre Selenium, Jenkins, Jira, Slack y la aplicación

La aplicación fue desarrollada en el framework Django en Python, lo cual permite trabajar de manera nativa con Selenium ya que cuenta con un módulo específico para Django. La librería contiene los métodos para especificar el paso a paso que debe realizar la prueba automatizada. Al crear las pruebas, estas deben ser monitoreadas por Jenkins, que será el agente encargado de testear el pipeline. Está integrado además con Slack y Jira para manejar la organización y división de tareas para el grupo, y reporta los resultados obtenidos en canales pertinentes para esto.

Es notable el esfuerzo que se elimina del equipo de trabajo al hacer uso de estas herramientas automatizadas que permiten desarrollar las pruebas de manera unitaria, verificar su rendimiento con un agente automatizado, y resolver cualquier posible problema asociado al chequear constantemente por el canal de comunicación.

Especificación y detalle de las pruebas

Se crearon pruebas de caso de uso con Selenium que siguen un flujo bien definido de acciones que varían desde introducir texto en los campos de la receta, seleccionar las opciones adecuadas del menú para realizar la acción deseada y verificar que se agregaron correctamente las recetas.

Para definir los casos de uso se codifican como procedimientos en Selenium, que tienen cada uno de los pasos indicados a realizar por la herramienta para verificar el cumplimiento de los requisitos.

Teniendo en cuenta lo anterior mencionado, se elaboraron casos de prueba que consideran formatos incorrectos, entradas vacías/campos sin rellenar,

límite de caracteres y también casos de éxito con respecto a las funciones definidas según los requerimientos funcionales.

Se busca probar estos casos para verificar que la aplicación soporta un uso inadecuado y que esta no genera fallas; en el fondo se busca cumplir con un software libre de errores.

Para los casos borde, estos están limitados al comportamiento de los campos de texto y su límite de caracteres. No hay funciones que hagan uso de comparadores lógicos, por lo que no se tienen más casos borde a probar.

Pruebas Automatizadas de Caso de Uso

Se definieron las siguientes pruebas a realizar con Selenium, se detallan en los flujos cada uno de los pasos realizados por la herramienta dentro de un Scenario y cuál es el resultado de las acciones realizadas.

Se realizan pruebas que tienen resultados válidos (como agregar correctamente una receta) y pruebas donde se falla de alguna forma en cumplir con el caso de uso, con el fin de observar el comportamiento de la aplicación con entradas inesperadas o no válidas. Las pruebas son codificadas de tal manera que buscan la persistencia en base de datos (en comparación a comprobar por HTML); adicionalmente, se nota que las pruebas hacen que la base de datos sea borrada con cada prueba unitaria, por lo que al hacer uso de una prueba que requiere, por ejemplo, una receta creada, se asume que la prueba primero creará la receta y luego ejecutará la prueba en cuestión.

El actor principal en estas pruebas es la herramienta Selenium que hará el papel de usuario y será referenciado como "Usuario final". Notar que se toma en cuenta las clases de equivalencia para las pruebas, lo que quiere decir que al probar para un campo de texto, se asume que el comportamiento de los otros campos de texto será análogo.

Nombre	Agregar receta		
ID	CU1 - test_CargaPagina		
Descripción	El usuario agrega una receta, la edita y por último la elimina		
Actores	Usuario final		
Precondiciones	Web app abierta en menú principal		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Añadir receta"	
	2	Escribe el nombre de la receta	
	3	Escribe la lista de ingredientes	
	4	Escribe los pasos a seguir	

	5	Sube una imagen de la receta preparada	
	6	Se escribe el autor de la receta	
	7	Se selecciona la opción "Subir Receta"	
Postcondiciones	El sistema muestra el mensaje de confirmación "Receta agregada satisfactoriamente!"		

Nombre	Visualizar detalles de una receta		
ID	CU2 - test_VerReceta		
Descripción	El usuario selecciona una receta del menú principal y puede visualizar los detalles de esta		
Actores	Usuario final		
Precondiciones	Web app abierta en página inicial y una receta agregada previamente		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Ver receta" de una receta disponible	
Postcondiciones	Se puede ver la vista detallada de la receta		

Nombre	Editar receta		
ID	CU3 - test_ActualizarReceta		
Descripción	El usuario selecciona una receta del menú y edita sus detalles		
Actores	Usuario final		
Precondiciones	Web app abierta en página inicial y una receta creada previamente		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Ver receta" de una receta disponible	
	2	En la vista de detalle selecciona la opción "Editar receta"	
	3	Modifica el nombre de la receta	
	4	Modifica los ingredientes	
	5	Modifica los pasos	
	6	Sube una nueva imagen	
	7	Selecciona la opción "Editar Receta"	

Postcondiciones	El sistema muestra el mensaje de confirmación "Receta editada satisfactoriamente!"
-----------------	--

Nombre	Eliminar una receta		
ID	CU4 - test_eliminarReceta		
Descripción	El usuario selecciona una receta del menú y la elimina		
Actores	Usuario final		
Precondiciones	Web app abierta en página inicial y una receta creada previamente		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Ver receta" de una receta disponible	
	2	En la vista de detalle selecciona la opción "Eliminar receta"	
Postcondiciones	El sistema vuelve a la vista de menú principal		

Nombre	Agregar una receta con el formato incorrecto - Ingredientes		
ID	CU5 - test_AddRecetaIncorrectaING		
Descripción	El usuario intenta agregar una receta con el campo ingredientes en formato incorrecto		
Actores	Usuario final		
Precondiciones	Web app abierta en página inicial		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Añadir receta"	
	2	Escribe el nombre de la receta	
	3	Escribe la lista de ingredientes - Falta al formato separado por saltos de línea	
	4	Escribe los pasos a seguir	
	5	Sube una imagen de la receta preparada	
	6	Se escribe el autor de la receta	
	7	Se selecciona la opción "Subir Receta"	
Postcondiciones	El sistema muestra el mensaje de advertencia "Completa este campo"		

Nombre	Agregar una receta con el formato incorrecto - Pasos		
ID	CU6 - test_AddRecetaIncorrectaPasos		
Descripción	El usuario intenta agregar una receta con el campo pasos en formato incorrecto		
Actores	Usuario final		
Precondiciones	Web app abierta en página inicial		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Añadir receta"	
	2	Escribe el nombre de la receta	
	3	Escribe la lista de ingredientes	
	4	Escribe los pasos a seguir - Falta al formato separado por saltos de línea	
	5	Sube una imagen de la receta preparada	
	6	Se escribe el autor de la receta	
	7	Se selecciona la opción "Subir Receta"	
Postcondiciones	El sistema muestra el mensaje de advertencia "Completa este campo"		

Nombre	Agregar una receta con falta de información - Imagen		
ID	CU7 - test_AddRecetaIncorrectaImg		
Descripción	El usuario intenta agregar una receta sin adjuntar una imagen		
Actores	Usuario final		
Precondiciones	Web app abierta en menú principal		
Flujos	No.	Flujo	Flujo alternativo
	1	Selecciona opción "Añadir receta"	
	2	Escribe el nombre de la receta	
	3	Escribe la lista de ingredientes	
	4	Escribe los pasos a seguir	
	5	Se escribe el autor de la receta	
	6	Se selecciona la opción "Subir Receta"	
Postcondiciones	El sistema muestra el mensaje de advertencia "Seleccione un archivo"		

Los scripts utilizados para realizar las pruebas descritas se pueden encontrar en el repositorio de Github en el archivo "SoftwareTesting-Integration-Process/recetario/mainApp/tests.py". Se puede ver el detalle de que chequea en cada instancia y prueba para poder replicar el trabajo en pruebas análogas.

Resultados de las pruebas

Luego de llevadas a cabo las pruebas especificadas a partir de la librería Selenium en Jenkins, esta nos entrega un reporte final, el cual indica que han sido pasadas todas las pruebas estructuradas, dicho reporte el cual puede ser observado dentro del repositorio del proyecto, el cual aloja los resultados actuales y anteriores de la ejecución.

La tabla 1 muestra los resultados obtenidos en cada parte del test, cuyo resumen se muestra en la figura 1.

Test Result : BaseTestCase

0 failures (±0)

7 tests (±0)

Took 36 sec.

 Add description

All Tests

Test name	Duration	Status
test_ActualizarReceta	10 sec	Passed
test_AddRecetaIncorrectaING	4.2 sec	Passed
test_AddRecetaIncorrectaImg	1.6 sec	Passed
test_AddRecetaIncorrectaPasos	4.6 sec	Passed
test_CargaPagina	4.5 sec	Passed
test_VerReceta	5.2 sec	Passed
test_eliminarReceta	5.6 sec	Passed

Tabla 1: Síntesis del reporte generado por Jenkins para comprobar estado del pipeline

 Build #29 (Dec 8, 2022, 9:52:50 PM)

No changes.

Started by user [PdS-GrupoGOD](#)**Revision:** d921e6a676a1a119c5401c77eb913f56098a73e1**Repository:** <https://github.com/MaxGGx/SoftwareTesting-Integration-Process.git>

- refs/remotes/origin/main

**Test Result** (no failures)

Figura 1: Resultado de la última prueba del build estable del pipeline

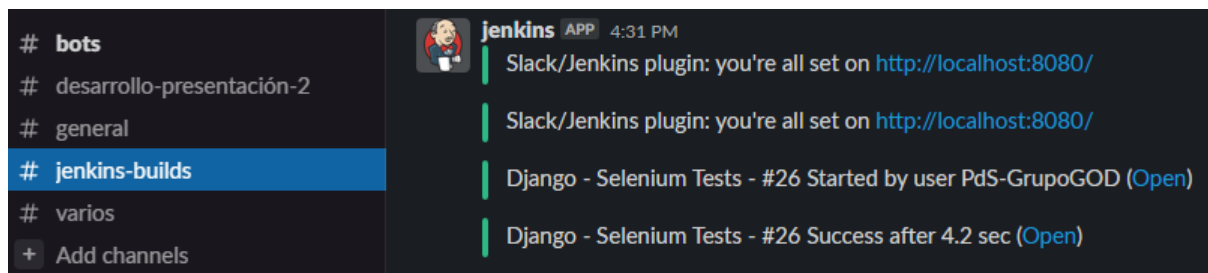


Figura 2: Integración de software de testing (Jenkins) con software de comunicación y organización (Slack) reportando resultados.

Mientras que en la figura 2 se muestra el proceso de testing siendo ejecutado por Jenkins de manera automatizada y comunicando al canal de Slack. Por último, estas tareas fueron divididas y asignadas desde Jira. La integración con Github, Slack y Jenkins permitió llevar control de los commits asociados a cada tarea y sus pruebas asociadas igualmente (figura 3).

Tablero STIP

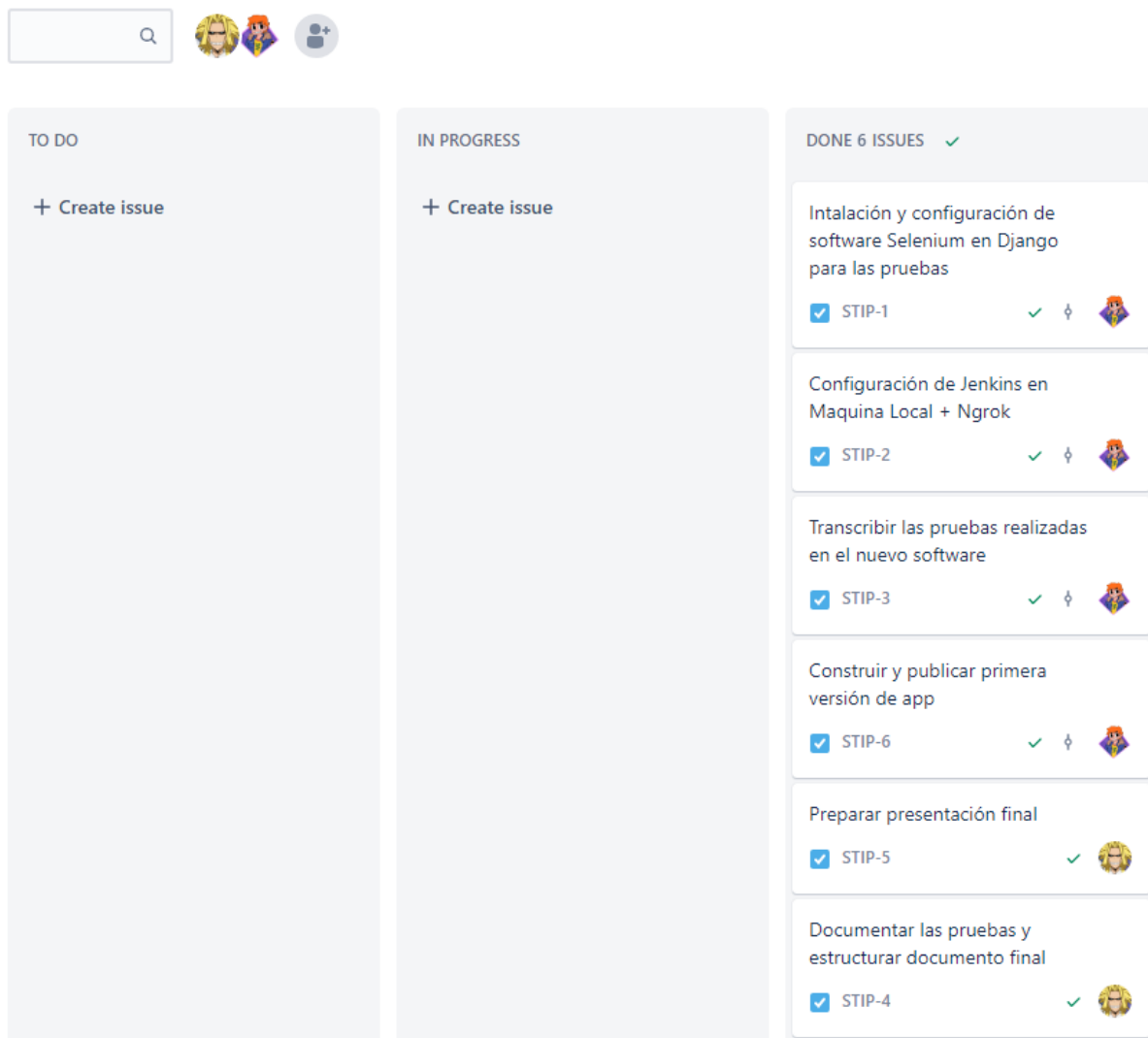


Figura 3: Tablero de Jira del proyecto

Anexos

Se deja anexo la configuración de jenkins:

General

Enabled 

Description

Jenkins automated test using Selenium on Django project

[Plain text] [Preview](#)

☐ Discard old builds ?

☒ GitHub project

Project url ?

<https://github.com/MaxGGx/SoftwareTesting-Integration-Process.git/>

Advanced...

☐ This project is parameterized ?

☐ Throttle builds ?

☐ Execute concurrent builds if necessary ?

Advanced...

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

<https://github.com/MaxGGx/SoftwareTesting-Integration-Process.git>

Credentials ?

- none -

+ Add

Advanced...

Add Repository



Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add ▾

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

H/15 * * * *

Would last have run at Friday, December 9, 2022 at 10:54:29 AM Chile Summer Time; would next run at Friday, December 9, 2022 at 11:09:29 AM Chile Summer Time.

☐ Ignore post-commit hooks ?

Build Environment

☐ Use secret text(s) or file(s) ?

☐ Run Xvnc during build ?

☐ With Ant ?

Build Steps

Custom Python Builder ?

Home ?

/mnt/c/Users/inaki/AppData/Local/Microsoft/WindowsApps/python3.exe

Nature ?

Shell

Command ?

```
. venv/bin/activate
sudo pip install seleniumbase
sudo pip list
python recetario/manage.py jenkins
```

Advanced...

Add build step ▾

Post-build Actions

Publish JUnit test result report ?

Test report XMLs

Fileset 'includes' setting that specifies the generated raw XML report files, such as 'myproject/target/test-reports/*.xml'. Basedir of the fileset is [the workspace root](#).

recetario/reports/*.xml

?

☐ Retain long standard output/error

Health report amplification factor ?

1.0

1% failing tests scores as 99% health. 5% failing tests scores as 95% health

Allow empty results ?

☐ Do not fail the build on empty test results

Skip publishing checks ?

☐ If unchecked, then issues will be published to SCM provider platforms



≡ **E-mail Notification** ?



Recipients

Whitespace-separated list of recipient addresses. May reference build parameters like \$PARAM. E-mail will be sent when a build fails, becomes unstable or returns to stable.

inaki.oyarzun@sansano.usm.cl

☒ Send e-mail for every unstable build



☐ Send separate e-mails to individuals who broke the build

≡ **Slack Notifications**



☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted